Databricks Fundamental

Complete Training Reference & Cheatsheet

Covers all 7 training modules · Based on official Databricks documentation
Sources: docs.databricks.com · spark.apache.org · delta.io

Modules Covered

M01 Platform & Workspace	**M02** Spark Architecture	**M03** ELT & Ingestion	**M04** Delta Lake
M05 Orchestration & Lakeflow	**M06** Unity Catalog	**M07** AI/BI & Genie	**+** External Connection

M01 · Platform & Workspace

docs.databricks.com/en/getting-started/overview.html

LAKEHOUSE ARCHITECTURE

A Databricks **Lakehouse** combines the openness and scalability of a data lake with the reliability and governance of a data warehouse. The three pillars are:

Layer	Technology	Purpose
Storage	Delta Lake (open format on cloud object store)	ACID transactions, versioning, schema enforcement
Compute	Apache Spark clusters, SQL Warehouses, Serverless	Batch, streaming, ML, SQL analytics
Governance	Unity Catalog	Fine-grained access control, lineage, discovery

COMPUTE OPTIONS

Type	Use case	Key property
All-Purpose Cluster	Interactive notebooks, development	Long-running; shared; billed when active
Job Cluster	Scheduled / automated jobs	Created on demand, terminated when job ends
SQL Warehouse	SQL Editor, BI tools (JDBC/ODBC)	Auto-suspend; optimised for concurrent SQL; supports Serverless
Serverless Compute	Notebooks, jobs, SQL (no cluster to manage)	Instant start; no cluster config; billed per DBU per second

KEY MAGIC COMMANDS

Command	Purpose
`%sql`	Run SQL in a Python/Scala cell
`%python`	Switch cell language to Python
`%sh`	Run shell commands on the driver
`%pip install <pkg>`	Install a Python library scoped to the notebook session
`%run ./path/notebook`	Execute another notebook inline (shares scope)
`%fs ls /`	List DBFS/Volume paths

WIDGETS (PARAMETERISATION)

```
dbutils.widgets.text("env", "dev", "Environment")
dbutils.widgets.dropdown("format", "delta", ["delta","parquet","csv"])
val = dbutils.widgets.get("env")
dbutils.widgets.removeAll()
```

✅ **Best practice:** Use Serverless compute for new development — instant start, no cluster configuration overhead, and cost-effective for intermittent workloads. docs

M02 · Spark Architecture

spark.apache.org/docs/latest/cluster-overview.html

DRIVER & EXECUTORS

Component	Role
Driver	Runs your application code; builds the DAG; schedules tasks; collects results. One per cluster.
Executor	Executes tasks in parallel; each holds a partition in memory/disk. Multiple per cluster.
Partition	A chunk of the dataset processed by one task on one executor. Default target: 128 MB.

LAZY EVALUATION & EXECUTION MODEL

Concept	Description
Transformation	Lazy — builds a new DataFrame without executing (e.g. `filter`, `select`, `join`, `groupBy`)
Action	Triggers execution — materialises results (e.g. `count()`, `show()`, `write()`, `collect()`)
Narrow transformation	No data movement across partitions (e.g. `filter`, `map`)
Wide transformation	Requires shuffle — redistributes data (e.g. `groupBy`, `join`, `distinct`)

DAG, STAGES & SHUFFLE

- Spark builds a **Directed Acyclic Graph (DAG)** of all transformations
- The DAG is split into **Stages** — boundaries occur at every wide transformation (shuffle)
- Each Stage is split into **Tasks** — one task per partition
- **Shuffle**: network transfer of data between executors — most expensive operation; avoid when possible

CATALYST OPTIMIZER & AQE

Feature	When	What it does
Catalyst	Compile-time	Transforms logical plan → physical plan; applies predicate pushdown, constant folding, column pruning
AQE (Adaptive Query Execution)	Runtime	Dynamically re-plans based on actual shuffle statistics: coalesces small partitions, changes join strategies, skew handling

> ✅ AQE is **enabled by default** in Databricks Runtime (Spark 3.0+). PySpark DataFrame API and `spark.sql()` generate an **identical execution plan** — there is no performance difference. docs

READING DATA — COMMON PATTERNS

```
# CSV — manual schema is faster for large files (avoids double-read)
df = (spark.read.format("csv")
  .option("header", True)
  .option("sep", ",")
  .option("nullValue", "NULL")
  .schema(my_schema)       # preferred over inferSchema=True in production
  .load("path/file.csv"))

# JSON (multi-line)
df = spark.read.option("multiLine", True).json("path/")

# Parquet / Delta (schema embedded — no options needed)
df = spark.read.parquet("path/")
df = spark.read.table("catalog.schema.table")
```

KEY DATAFRAME TRANSFORMATIONS

Operation	Syntax
Select columns	`df.select("col1", "col2")`
Add / modify column	`df.withColumn("new", col("x") * 2)`
Rename column	`df.withColumnRenamed("old", "new")`
Filter rows	`df.filter(col("age") > 18)`
Conditional logic	`when(col("x") > 0, "pos").otherwise("neg")`
Drop column	`df.drop("col")`
Cast type	`df.withColumn("id", col("id").cast("integer"))`
Deduplicate	`df.dropDuplicates(["col1", "col2"])`
Sort	`df.orderBy(col("date").desc())`
Handle nulls	`df.fillna({"col": 0}) / df.dropna()`

AGGREGATION & GROUPBY

```
from pyspark.sql.functions import count, sum, avg, min, max
df.groupBy("region").agg(
    count("*").alias("orders"),
    sum("amount").alias("revenue"),
    avg("amount").alias("avg_order")
).filter(col("orders") > 100)   # HAVING equivalent
```

JOINS

Type	Rows returned
`inner`	Only rows matching in BOTH tables
`left`	All left rows + matching right rows (nulls for non-matches)
`right`	All right rows + matching left rows
`full` / `outer`	All rows from both tables
`cross`	Cartesian product — all combinations

```
result = df_orders.join(df_customers, df_orders.customer_id == df_customers.id, "left")
```

TEMPORARY VIEWS & SQL

```
df.createOrReplaceTempView("orders_view")
spark.sql("SELECT region, SUM(amount) FROM orders_view GROUP BY region").show()
```

JSON OPERATIONS

```
from pyspark.sql.functions import explode, get_json_object, from_json
df.withColumn("item", explode(col("items_array")))   # array → rows
df.withColumn("city", get_json_object(col("json_str"), "$.address.city"))
```

M04 · Delta Lake — Fundamentals

docs.delta.io · docs.databricks.com/en/delta/index.html

WHAT IS DELTA LAKE?

Delta Lake is an open-source storage layer that brings **ACID transactions**, scalable metadata handling, and unified batch/streaming to data lakes. It stores data as **Parquet files** + a JSON-based **transaction log** (`_delta_log/`) that records every change.

CORE ACID GUARANTEES

Property	Meaning
Atomicity	Each write either fully succeeds or fully fails — no partial writes
Consistency	Schema and constraints are enforced on every write
Isolation	Concurrent reads and writes do not corrupt each other (optimistic concurrency)
Durability	Committed data is permanently stored even after failures

CRUD OPERATIONS

```
-- INSERT
INSERT INTO catalog.schema.orders VALUES (1001, 'C001', 99.99, current_date());

-- UPDATE
UPDATE catalog.schema.orders SET status = 'shipped' WHERE order_id = 1001;

-- DELETE
DELETE FROM catalog.schema.orders WHERE status = 'cancelled';

-- MERGE (upsert) — most powerful pattern
MERGE INTO target t USING source s ON t.id = s.id
  WHEN MATCHED THEN UPDATE SET t.val = s.val
  WHEN NOT MATCHED THEN INSERT *
  WHEN NOT MATCHED BY SOURCE THEN DELETE;
```

TIME TRAVEL

```
-- By version number
SELECT * FROM my_table VERSION AS OF 5;

-- By timestamp
SELECT * FROM my_table TIMESTAMP AS OF '2026-01-15 08:00:00';

-- In PySpark
df = spark.read.option("versionAsOf", 5).table("my_table")

-- View history
DESCRIBE HISTORY my_table;
```

VACUUM — STORAGE CLEANUP

```
-- Default retention: 7 days (168 hours). Files older than retention are deleted.
VACUUM my_table;                      -- uses default retention
VACUUM my_table RETAIN 168 HOURS;     -- explicit

-- DRY RUN first (check what would be deleted — no actual deletion)
VACUUM my_table DRY RUN;
```

> ⚠️ Do not set retention below 7 days on active tables — concurrent readers may still need older files. Vacuumed versions are permanently inaccessible for Time Travel.

OPTIMIZE & Z-ORDER

```
OPTIMIZE my_table;                          -- compact small files
OPTIMIZE my_table ZORDER BY (order_date);  -- compact + co-locate by column
```

Z-ORDER clusters related data in the same files, dramatically improving filter performance on the specified columns. Best applied to high-cardinality filter columns. docs

MANAGED VS EXTERNAL TABLES

	Managed	External
Data location	Databricks-managed storage (Unity Catalog default)	User-specified cloud path (ABFSS, S3, GCS)
DROP TABLE	Deletes both metadata AND data files	Deletes only metadata; data files remain
Use case	Default — full lifecycle management by Databricks	Bring-your-own-storage; existing data pipelines

M05 · Orchestration & Lakeflow

docs.databricks.com/en/workflows/index.html

DATABRICKS WORKFLOWS (JOBS) — KEY CONCEPTS

Concept	Description
Job	A configured execution unit containing one or more tasks
Task	A single unit of work: Notebook, Python script, SQL, dbt, Delta Live Tables, etc.
Task dependency	Defines execution order (DAG of tasks within a job)
Repair Run	Reruns only failed tasks and their downstream dependencies; skips successful tasks
Job cluster	Ephemeral cluster created at job start, terminated at end — cost-efficient for production

TRIGGER TYPES

Trigger	When to use
Scheduled (CRON)	Time-based recurring runs
File Arrival	Cloud storage path monitoring — triggers when new files land
Continuous	Job runs immediately after the previous run completes
Manual	On-demand / API-triggered runs

CRON SYNTAX (QUARTZ FORMAT)

```
┌──────── seconds (0-59)
│  ┌───── minutes (0-59)
│  │  ┌── hours (0-23)
│  │  │  ┌── day of month (1-31)
│  │  │  │  ┌── month (1-12 or JAN-DEC)
│  │  │  │  │  ┌── day of week (1-7 or SUN-SAT)
│  │  │  │  │  │  ┌ year (optional)
0  30  2  *  *  ?     → every day at 02:30
0  0   8  ?  *  MON-FRI → weekdays at 08:00
0  0   0  1  *  ?     → 1st of every month at midnight
```

LAKEFLOW PIPELINES (DELTA LIVE TABLES)

Object type	Description
Streaming Table	Incrementally ingests data from a streaming or batch source; processes only new data each run
Materialized View	Computes and caches query results; refreshed on each pipeline run (full recompute)
View	Logical intermediate transformation; not stored; recomputed when referenced

```
-- Minimal DLT pipeline (SQL)
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS SELECT * FROM STREAM(LIVE.raw_landing);

CREATE OR REFRESH MATERIALIZED VIEW silver_orders
AS SELECT order_id, customer_id, amount FROM LIVE.bronze_orders WHERE amount > 0;
```

> ✅ **Tip:** It's worth knowing CRON syntax and File Arrival trigger configuration — these are the most frequently used trigger modes in production environments. docs

MONITORING VIA SYSTEM TABLES

```
-- Job run history (requires system schema access)
SELECT * FROM system.lakeflow.job_run_timeline
```

```
WHERE result_state = 'FAILED'
  AND period_start_time >= current_timestamp() - INTERVAL 7 DAYS;
```

M06 · Unity Catalog & Governance

docs.databricks.com/en/data-governance/unity-catalog/index.html

OBJECT HIERARCHY (3-LEVEL NAMESPACE)

```
Metastore (one per region — workspace-agnostic)
  └ Catalog          (e.g. dev, prod, sandbox)
      └ Schema       (logical grouping — equivalent to database)
          └ Table / View / Volume / Function
```

```
-- Always use fully-qualified names in production:
SELECT * FROM main.retail.orders;
--           catalog.schema.table
```

GRANTING PERMISSIONS — TOP-DOWN HIERARCHY

```
-- You must grant each level before the next level is accessible:
GRANT USE CATALOG ON CATALOG main TO `user@company.com`;
GRANT USE SCHEMA  ON SCHEMA  main.retail TO `user@company.com`;
GRANT SELECT      ON TABLE   main.retail.orders TO `user@company.com`;

-- Revoke
REVOKE SELECT ON TABLE main.retail.orders FROM `user@company.com`;
```

Privilege	Object level	What it allows
`USE CATALOG`	Catalog	See/access the catalog (required first step)
`USE SCHEMA`	Schema	See/access the schema
`SELECT`	Table / View	Read data
`MODIFY`	Table	Write / update / delete data
`CREATE TABLE`	Schema	Create new tables in the schema
`ALL PRIVILEGES`	Any	All permissions — use only for admin/dev

EXTERNAL CONNECTIVITY

Object	Purpose
Storage Credential	Stores a cloud identity (Access Connector / IAM role) for authenticating to external storage
External Location	Maps an ABFSS/S3/GCS path to a Storage Credential — enables `READ/WRITE FILES`
External Table	Delta/Parquet table whose data files reside in an External Location

DELTA SHARING (OPEN PROTOCOL)

Object	Description
Share	A named container of tables/schemas to be shared
Recipient	An external entity (person/organisation) that receives access
Provider	The Databricks workspace sharing the data

```
CREATE SHARE my_share;
ALTER SHARE my_share ADD TABLE main.retail.orders;
CREATE RECIPIENT external_partner;   -- generates activation link
```

> ✅ Recipients do **not** need Databricks to consume a Delta Share — they can use the open Delta Sharing client (Python, Spark, pandas, Power BI). docs

DATA LINEAGE

Unity Catalog automatically captures **table-level and column-level lineage** for all queries and jobs run against Unity Catalog tables. View in: **Catalog Explorer** → **Table** → **Lineage tab**.

M07 · AI/BI Dashboards & Genie

docs.databricks.com/en/dashboards/index.html

AI/BI DASHBOARDS

- Interactive visual analytics built on Unity Catalog data
- Chart types: bar, line, area, scatter, pie, counter (KPI tile), table, heatmap
- Filters: date pickers, dropdowns, cross-filtering between widgets
- Publish = shareable URL; viewers do *not* need to run queries
- Scheduled email delivery supported
- Can embed in external applications via iframe

Docs: docs.databricks.com/en/dashboards

AI/BI GENIE SPACE

- Natural-language interface: ask questions in plain English → Genie generates & runs SQL
- Backed by Unity Catalog tables configured in the Space
- Supports follow-up questions (conversational context)
- Answers can be saved as dashboard widgets
- Best for: ad-hoc exploration, self-service analytics
- Limitations: complex multi-step logic, highly formatted reports

Docs: docs.databricks.com/en/genie

WHEN TO USE WHICH TOOL

Scenario	Best tool
Approved KPI dashboard for the business (refreshes daily)	AI/BI Dashboard
Ad-hoc: "What were our top 10 products last month?"	Genie Space
Exploration before building a dashboard	SQL Editor
Share live data with external teams (no Databricks account)	Delta Sharing + Dashboard URL

➕ External Connection — ADLS Gen2

docs.databricks.com/en/connect/unity-catalog/external-locations.html

SETUP FLOW (5 STEPS)

Step	Where	What
1	Azure Portal → Managed Resource Group	Copy **Resource ID** of Access Connector (`Microsoft.Databricks/accessConnectors/<name>`)
2	ADLS Gen2 → Access Control (IAM)	Assign `Storage Blob Data Contributor` role to Managed Identity (Access Connector)
3	Databricks Catalog → External Data → Credentials	Create Storage Credential with Connector Resource ID
4	Databricks Catalog → External Locations	Create External Location with ABFSS URL + Storage Credential
5	External Location → Permissions	Grant `READ FILES` / `WRITE FILES` / `CREATE EXTERNAL TABLE`

```
-- Verify External Location from SQL
SHOW EXTERNAL LOCATIONS;
LIST 'abfss://container@account.dfs.core.windows.net/';
```

⚠️ Use `Storage Blob Data Reader` in production (not Contributor). Apply granular UC permissions — never `ALL PRIVILEGES` in production.

Quick Reference — Most Important SQL Commands

DELTA LAKE

```
DESCRIBE DETAIL table;
DESCRIBE HISTORY table;
OPTIMIZE table ZORDER BY (col);
VACUUM table RETAIN 168 HOURS;
RESTORE TABLE t TO VERSION AS OF 3;

-- Time Travel
SELECT * FROM t VERSION AS OF 2;
SELECT * FROM t TIMESTAMP AS OF '...';
```

UNITY CATALOG

```
SHOW CATALOGS; SHOW SCHEMAS IN catalog;
SHOW TABLES IN catalog.schema;
DESCRIBE TABLE EXTENDED cat.sch.tbl;

GRANT SELECT ON TABLE cat.sch.tbl TO `u`;
REVOKE SELECT ON TABLE cat.sch.tbl FROM `u`;
SHOW GRANTS ON TABLE cat.sch.tbl;
```

Official Documentation Links

Topic	URL
Databricks documentation home	https://docs.databricks.com
Delta Lake open source	https://docs.delta.io
Apache Spark documentation	https://spark.apache.org/docs/latest/
Databricks Workflows / Jobs	https://docs.databricks.com/en/workflows/index.html
Unity Catalog overview	https://docs.databricks.com/en/data-governance/unity-catalog/index.html
Delta Sharing	https://docs.databricks.com/en/delta-sharing/index.html
AI/BI Dashboards	https://docs.databricks.com/en/dashboards/index.html
AI/BI Genie	https://docs.databricks.com/en/genie/index.html
External Locations & Storage Credentials	https://docs.databricks.com/en/connect/unity-catalog/external-locations.html
Lakeflow Pipelines (DLT)	https://docs.databricks.com/en/dlt/index.html
Serverless Compute	https://docs.databricks.com/en/compute/serverless.html
Adaptive Query Execution	https://docs.databricks.com/en/optimizations/aqe.html
OPTIMIZE and Z-ORDER	https://docs.databricks.com/en/delta/optimize.html